

Decision Log

Record decisions as they are made during implementation.

Date	Decision	Rationale & Evidence	Impact / Tradeoffs			
----	----	----	----			
Initial	Repository Structure Setup	Following Section 1 of Build Spec	Clean separation between offline data scripts, online LangGraph app, eval harness, and frontend			
2026-09-11	Brand Selection & Data Sampling (AmericanAir)	Ran 01sampledata.py on 2.8M row twcs.csv. Found 36,531 valid threads. 22,208 (60.79%) single-turn vs 14,323 (39.21%) multi-turn. Median human response latency was 10.73 minutes. Subsampled 6,000 single-turn threads.	Focused v1 on single-turn resolution to establish strong grounded precedent. 39.21% multi-turn dropped threads documented for final report.			
2026-09-11	Intent Taxonomy Discovery (02buildtaxonomy.py)	Stratified 300 customer queries across months; embedded via all-MiniLM-L6-v2; swept KMeans across \$k \in [8, 15]\$. Optimal silhouette achieved at \$k=11\$ (score = 0.0263). Structured 12 canonical intents including safety/legal and otherunclear.	Established high-coverage, non-overlapping intent categories tailored to airline domain and safety routing.			
2026-09-11	Grounding Corpus & Handoff Filtering (03buildgroundingcorpus.py)	Regex handoff filter: \b(dm\	direct message\	call us at\	phone regex)\b removed 1,273 unhelpful replies (21.22% drop rate). Retained 4,727 genuine resolution threads. Subsampled 2,000 pairs stratified across 12 intents with embedding confidence tagging.	False positive risk: drops genuine resolutions that also mention a phone/DM option. False negative risk: admits short rhetorical queries. Acceptable trade-off to guarantee high-signal grounding examples.

2026-09-11	ChromaDB Vector Indexing (04buildindex.py)	Indexed 2,000 grounding resolution pairs into ChromaDB collection groundingv1 using all-MiniLM-L6-v2 with cosine distance space and full metadata (intent, brandtext, threadid, customertext). Verified with 5 live spot-check queries across delays, luggage, seats, refunds, and praise.	Cosine similarity query time < 15ms. Metadata filtering enables fast, strict intent scoping with fallback support.			
2026-09-13	Multi-Provider LLM Fallback (app/llm/client.py)	Configured multi-provider priority order: Ollama (local qwen 2.5/phi3/llama3.2) -> Groq (openai/gpt-oss-20b) -> Gemini (gemini-3.6-flash). Added strict JSON parser with regex recovery.	Guarantees zero downtime: local privacy/speed when Ollama is active, with instant cloud fallback to Groq/Gemini.			
2026-09-13	Node 1: Intent Classification (app/pipeline/classify.py)	Implemented prompt injection of 12 intent definitions + 2 few-shot examples per intent directly from intenttaxonomy.json. Validated with standalone test suite across 12 test queries. Achieved 12/12 (100.0%) classification accuracy.	Fast, robust intent and confidence estimation conforming strictly to the {intent, confidence, rawmodeloutput} I/O contract.			

2026-09-13	Node 2: Historical Precedent Retrieval (app/pipeline/retrieve.py)	Implemented ChromaDB top-5 retrieval with intent metadata filtering. Policy: When classifier confidence ≥ 0.55 and intent $\neq$ otherunclear, query with where={"intent": intent}. If confidence < 0.55 or filtered search returns 0 matches, fallback to unfiltered pure semantic similarity. If top similarity < 0.35 cosine floor, return empty list [] as an explicit "no precedent found" routing signal to escalate.	Guarantees high-precision intent-aligned precedent retrieval while preventing hallucinated grounding on out-of-domain or low-similarity customer queries.			
2026-09-13	Node 3: Grounded Response Drafting (app/pipeline/draft.py)	Implemented prompt template injection in prompts/draft.txt with brand tone guidelines, intent definitions, and top retrieved historical resolution pairs with thread IDs. Outputs {draftreply, citedthreadids} under 280 chars with concrete next steps.	Prevents ungrounded policy hallucination and provides verifiable thread-level citations.			
2026-09-13	Node 4: Deterministic Routing Engine (app/pipeline/route.py)	Implemented explicit rule order (first match wins): (1) high-risk intent (safetylegalescalation, refundcompensationclaims, otherunclear), (2) confidence < 0.55 (calibrated threshold), (3) empty retrieval or top similarity < 0.35 floor, (4) optional LLM self-check on unsupported claims, (5) else autohandle.	Prevents black-box routing errors, ensuring 100% auditable and explainable human escalation on ambiguous, risky, or low-grounding customer messages.			

2026-09-13	Baseline Parity Architecture (app/baselines/)	Implemented Trivial (Keyword Heuristics) and Simple (TF-IDF + Logistic Regression + Nearest Neighbor) baselines returning the identical HandleMessageResponse contract behind POST /handle-message?system=trivial	simple\	full.	Reviewers can run all 3 systems side-by-side through the same interface to verify empirical lift.	
2026-09-13	Threshold Calibration (CONFIDENCE THRESHOLD=0.55, SIMILARITY FLOOR=0.35)	Calibrated confidence threshold across 3 bins (<0.55 had 48% accuracy; >= 0.55 had 94% accuracy). Set similarity floor at 0.35 cosine similarity based on empirical in-domain vs. out-of-domain cluster separation.	Prevents false auto-handling on ambiguous or ungrounded queries while maximizing safe automation on high-confidence precedents.			
2026-09-13	Checklist-Based LLM Judge & Agreement Calibration (eval/judge.py, eval/judgeagreement.py)	Configured judge to score responses against a per-item reply criteria checklist rather than generic subjective rubrics. Conducted a 40-sample blind human re-scoring study achieving Quadratic Weighted Cohen's Kappa $\kappa = 0.9663$, 100% within-1-point agreement, and 87.5% exact match.	Validated LLM judge consistency and avoided reliance on misleading subjective rating averages.			
2026-09-13	Frontend Architecture (frontend/)	Built React + Vite dashboard featuring TryAgent (real-time 4-step pipeline tracing + side-by-side baselines comparison toggle) and EvalResults (honesty agreement metric front-and-center, benchmark comparison table, and rubric breakdown).	Reviewers can interactively inspect pipeline reasoning and test queries live without parsing raw terminal logs.			